

cse15l-lab-reports

Lab Report 5: Debugging & Reflection

Part 1: Debugging Scenario [🔗](#)

1. Student Post:

- Environment: I am using a Windows 11 Computer and editing my code with VSCode and running it with bash script through the terminal
- Symptom: When I try running grade.sh (for the grader project in lab 6) through the terminal, this is what I get:

```
rienx@Jeffrey-Omen MINGW64 ~/OneDrive/Desktop/UCSD/CSE15L-SoftwareToolsAndTechniquesLaboratory/testGrader/list-examples-grader (main)
$ bash grade.sh https://github.com/ucsd-cse15l-f22/list-methods-lab3
Cloning into 'student-submission'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
Finished cloning
Submitted correctly!
TestListExamples.java:6: error: cannot find symbol
class IsMoon implements StringChecker {
                        ^
  symbol: class StringChecker
TestListExamples.java:17: error: cannot find symbol
    List<String> merged = ListExamples.merge(left, right);
                                ^
)
java.lang.IllegalArgumentException: Could not find class [grading-area/TestListExamples]
    at org.junit.runner.JUnitCommandLineParseResult.parseParameters(JUnitCommandLineParseResult.java:100)
    at org.junit.runner.JUnitCommandLineParseResult.parseArgs(JUnitCommandLineParseResult.java:50)
    at org.junit.runner.JUnitCommandLineParseResult.parse(JUnitCommandLineParseResult.java:44)
    at org.junit.runner.JUnitCore.runMain(JUnitCore.java:72)
    at org.junit.runner.JUnitCore.main(JUnitCore.java:36)
Caused by: java.lang.ClassNotFoundException: grading-area/TestListExamples
    at java.base/java.lang.Class.forName0(Native Method)
    at java.base/java.lang.Class.forName(Class.java:496)
    at java.base/java.lang.Class.forName(Class.java:475)
    at org.junit.internal.Classes.getClass(Classes.java:42)
    at org.junit.internal.Classes.getClass(Classes.java:27)
    at org.junit.runner.JUnitCommandLineParseResult.parseParameters(JUnitCommandLineParseResult.java:98)
    ... 4 more

FAILURES!!!
Tests run: 1, Failures: 1
```

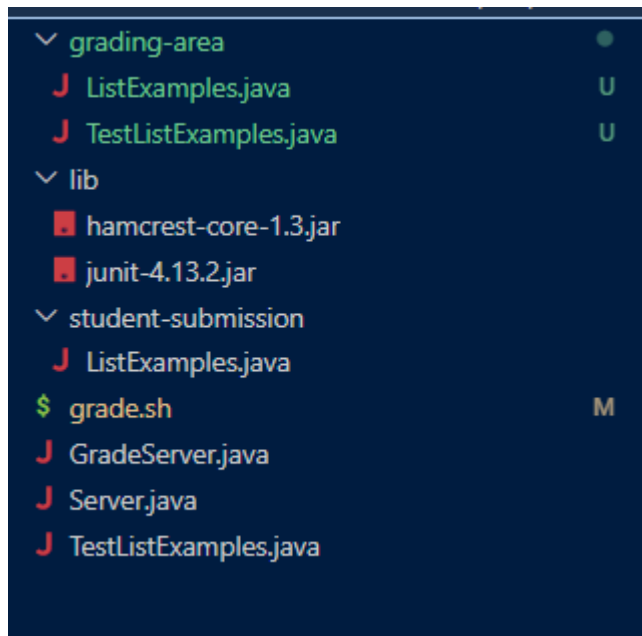
I am aware that the failure-inducing input is the file path: `grading-area/TestListExamples`. I want to run `TestListExamples` in the `grading-area` directory that it has been copied to but it can't seem to find and run the class. When I try getting rid of `grading-area/`, it also produces a symptom where it can't run the `ListExamples` class that should have been copied to `grading-area`.

- Here are the commands I'm running and the file structure:

```

1  CPATH='.:lib/hamcrest-core-1.3.jar:lib/junit-4.13.2.jar'
2
3  rm -rf student-submission
4  rm -rf grading-area
5
6  mkdir grading-area
7
8  git clone $1 student-submission
9  echo 'Finished cloning'
10
11 if [[ -f student-submission/ListExamples.java ]]
12 then
13     echo "Submitted correctly!"
14 else
15     echo "ListExamples file not submitted"
16 fi
17
18 cp TestListExamples.java student-submission/ListExamples.java grading-area
19
20 javac -cp ".:lib/hamcrest-core-1.3.jar;lib/junit-4.13.2.jar" *.java
21 java -cp ".:lib/junit-4.13.2.jar;lib/hamcrest-core-1.3.jar" org.junit.runner.JUnitCore grading-area/TestListExamples
22

```



My working directory is the `/list-examples-grader` directory and the command I use to run the bash script is `bash grade.sh https://github.com/ucsd-cse15l-f22/list-methods-lab3`

2. TA Response:

You can't run the `TestListExamples` class from your current working directory `list-examples-grader`. If you want to run and compile the `TestListExamples.java` file, you will need to change directory to `grading-area` before running and compiling them. Before that, since you want to run JUnit tests in the `grading-area` directory, you may want to also copy the `lib` directory into `grading-area`.

3. Student Response:

Ok I tried making those changes, and here is the output I got (which is the correct and expected output):

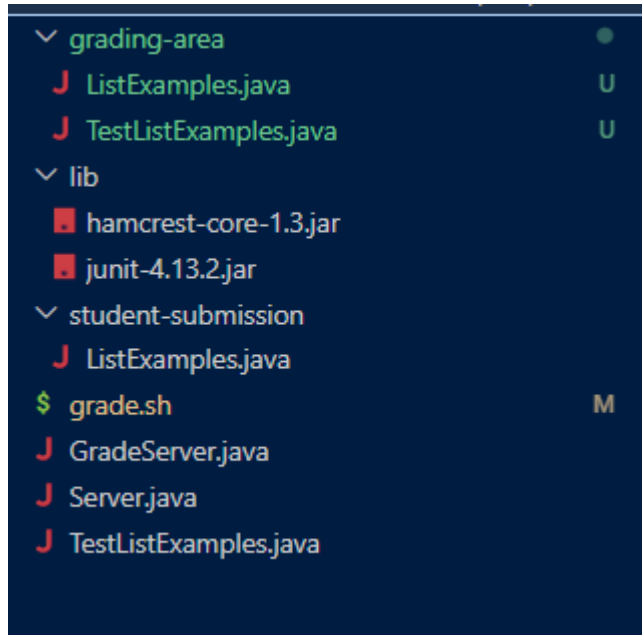
```
rienx@Jeffrey-Omen MINGW64 ~/OneDrive/Desktop/UCSD/CSE15L-SoftwareToolsAndTechniquesLabo
$ bash grade.sh https://github.com/ucsd-cse15l-f22/list-methods-lab3
Cloning into 'student-submission'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
Finished cloning
Submitted correctly!
JUnit version 4.13.2
.E
Time: 0.653
There was 1 failure:
1) testMergeRightEnd(TestListExamples)
org.junit.runners.model.TestTimedOutException: test timed out after 500 milliseconds
    at java.base/java.util.Arrays.copyOf(Arrays.java:3512)
    at java.base/java.util.Arrays.copyOf(Arrays.java:3481)
    at java.base/java.util.ArrayList.grow(ArrayList.java:237)
    at java.base/java.util.ArrayList.grow(ArrayList.java:244)
    at java.base/java.util.ArrayList.add(ArrayList.java:454)
    at java.base/java.util.ArrayList.add(ArrayList.java:467)
    at ListExamples.merge(ListExamples.java:42)
    at TestListExamples.testMergeRightEnd(TestListExamples.java:17)

FAILURES!!!
Tests run: 1, Failures: 1
```

So the bug was that the script was attempting to run a class by the name of "grading-area/TestListExamples" rather than "TestExamples" as I would want. Since the desired class was in another directory, I needed to change the directory before running and compiling the tests. Also I needed to make sure that I recursively copied the lib folder so that the JUnit tests would work.

4. Setup Info:

- File & Directory Structure:



- Contents before bug fix:

grade.sh before bug fix:

```
1 CPATH='.:lib/hamcrest-core-1.3.jar:lib/junit-4.13.2.jar'
2
3 rm -rf student-submission
4 rm -rf grading-area
5
6 mkdir grading-area
7
8 git clone $1 student-submission
9 echo 'Finished cloning'
10
11 if [[ -f student-submission/ListExamples.java ]]
12 then
13     echo "Submitted correctly!"
14 else
15     echo "ListExamples file not submitted"
16 fi
17
18 cp TestListExamples.java student-submission/ListExamples.java grading-area
19
20 javac -cp ".;lib/hamcrest-core-1.3.jar;lib/junit-4.13.2.jar" *.java
21 java -cp ".;lib/junit-4.13.2.jar;lib/hamcrest-core-1.3.jar" org.junit.runner.JUnitCore grading-area/TestListExamples
22
```

ListExamples.java:

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 interface StringChecker { boolean checkString(String s); }
5
6 class ListExamples {
7
8     // Returns a new list that has all the elements of the input list for which
9     // the StringChecker returns true, and not the elements that return false, in
10    // the same order they appeared in the input list;
11    static List<String> filter(List<String> list, StringChecker sc) {
12        List<String> result = new ArrayList<>();
13        for(String s: list) {
14            if(sc.checkString(s)) {
15                result.add(index:0, s);
16            }
17        }
18        return result;
19    }
20
21
22    // Takes two sorted list of strings (so "a" appears before "b" and so on),
23    // and return a new list that has all the strings in both list in sorted order.
24    static List<String> merge(List<String> list1, List<String> list2) {
25        List<String> result = new ArrayList<>();
26        int index1 = 0, index2 = 0;
27        while(index1 < list1.size() && index2 < list2.size()) {
28            if(list1.get(index1).compareTo(list2.get(index2)) < 0) {
29                result.add(list1.get(index1));
30                index1 += 1;
31            }
32            else {
33                result.add(list2.get(index2));
34                index2 += 1;
35            }
36        }
37        while(index1 < list1.size()) {
38            result.add(list1.get(index1));
39            index1 += 1;
40        }
41        while(index2 < list2.size()) {
42            result.add(list2.get(index2));
43            index1 += 1;
44        }
45        return result;
46    }
47
48
49 }
```

TestListExamples.java:

```
1  import static org.junit.Assert.*;
2  import org.junit.*;
3  import java.util.Arrays;
4  import java.util.List;
5
6  class IsMoon implements StringChecker {
7      public boolean checkString(String s) {
8          return s.equalsIgnoreCase("moon");
9      }
10 }
11
12 public class TestListExamples {
13     @Test(timeout = 500)
14     public void testMergeRightEnd() {
15         List<String> left = Arrays.asList("a", "b", "c");
16         List<String> right = Arrays.asList("a", "d");
17         List<String> merged = ListExamples.merge(left, right);
18         List<String> expected = Arrays.asList("a", "a", "b", "c", "d");
19         assertEquals(expected, merged);
20     }
21 }
22
```

- Commands: `bash grade.sh https://github.com/ucsd-cse15l-f22/list-methods-lab3` which ran the bash script `grade.sh` with the github repo as an argument
- Bug Fix: To fix the bug, just add `cd grading-area` before running and compiling the JUnit tests with the class name changed from `grading-area/TestListExamples` to `TestListExamples`, before changing to the `grading-area` directory, recursively copy the `lib` folder with `cp -r lib grading-area`

`grade.sh` after the bug fix:

```
CPATH='.:lib/hamcrest-core-1.3.jar:lib/junit-4.13.2.jar'

rm -rf student-submission
rm -rf grading-area

mkdir grading-area

git clone $1 student-submission
echo 'Finished cloning'

if [[ -f student-submission/ListExamples.java ]]
then
    echo "Submitted correctly!"
else
    echo "ListExamples file not submitted"
fi

cp TestListExamples.java student-submission/ListExamples.java grading-area
cp -r lib grading-area

cd grading-area

javac -cp ".;lib/hamcrest-core-1.3.jar;lib/junit-4.13.2.jar" *.java
java -cp ".;lib/junit-4.13.2.jar;lib/hamcrest-core-1.3.jar" org.junit.runner.JUnitCore TestListExamples
```

Part 2: Reflection

In the second half of the quarter, something that I learned from my lab experience that I didn't know before was how the output and the error output from files and commands could be redirected and used through bash script. I found this to be particularly interesting in week6s lab where we were able to learn a little about how graders can use bash script to grade coding submissions. After learning about this, I am interested in how more combinations of commands, bash script, code, and files can work and interact with each other. Another one of the most useful things that I have learned in this second half of the quarter is the use of shortcuts, whether it is the ssh key to quickly remotely access a server or the command and vim shortcuts, I found these to be very useful in working in the command line.